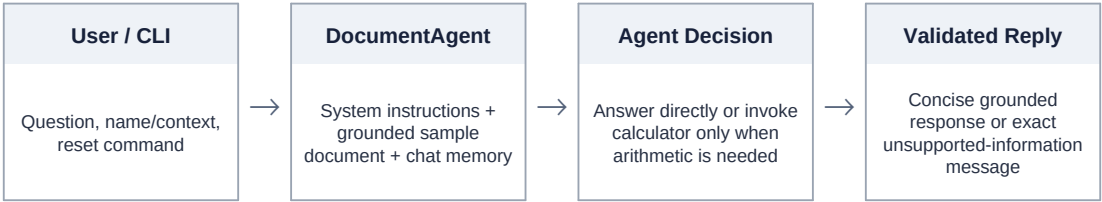


PointStar Developer Intern Assessment - Part 3

Document Q&A Agent - Architecture, Trade-offs & Failure Handling
Candidate: Aw Jia Yi | Python + Google GenAI SDK

1. Architecture



Grounding & Memory	Tool Use	Reliability
The sample handbook is loaded at startup and inserted into the system instruction. Company-policy answers must use only document facts. The Gemini chat keeps short-term conversation context, such as the user's name.	A safe Python calculator is exposed as a function tool. Gemini uses automatic function calling and should invoke the tool only when arithmetic is required; ordinary document questions are answered without a tool call.	Temporary 503 failures use bounded exponential-backoff retries. A 429 quota/rate-limit condition can trigger a fallback model. Successful chat history is preserved when a new chat session is created.

2. Trade-offs and Failure Points

TRADE-OFFS	FAILURE POINTS / MITIGATIONS
• Direct document injection vs. RAG: Chosen for one small sample document. It is simple, transparent and easy to demo, but would not scale well to a large knowledge base. • Session memory vs. persistent memory: Chat history is sufficient for the assessment, but it is intentionally not stored across application restarts. • Automatic function calling: Reduces custom orchestration code and demonstrates agentic tool selection, while relying on the model/SDK to make the tool-use decision.	• API outage / high demand: 503 errors are retried with bounded exponential backoff. • Quota / rate limit: 429 is handled separately so the agent can attempt the fallback model rather than repeatedly retrying the same exhausted model. • Invalid key or model configuration: Client errors fail fast with logs instead of being silently retried. • Missing document fact: The agent is instructed to return exactly: "The provided document does not contain this information."

3. Local Setup & Validation

Setup: <code>pip install -r requirements.txt copy .env.example to .env and add GEMINI_API_KEY python main.py</code> Tests: <code>python -m pytest -q</code> Manual validation covers memory, document Q&A, calculator tool use, unsupported-information guardrail, memory reset, retry behavior and fallback behavior.
--

Design goal: Keep the implementation small enough to explain in an interview while still demonstrating grounding, short-term memory, agentic tool selection, logging, testing and operational resilience.